Python for Artificial Intelligence

NumPy: Arrays for Numerical Computing

Week 3, Day 1

Today's Agenda

- Hour 1: Creating arrays, dimensions, shape, and indexing
- Hour 2: Slicing, vectorized operations, and broadcasting
- Hour 3: Array manipulation practice, and why AI data is represented as matrices
- By the end: you can create, inspect, and manipulate NumPy arrays instead of looping over plain Python lists

Hour 1

Arrays, Dimensions & Indexing

CONCEPT

Why Do We Need NumPy?

- A Python list can hold numbers, but math on it means writing a loop for every operation
- NumPy (Numerical Python) gives you the array: a fixed-type grid of numbers built for fast math
- Every AI and data science library (Pandas, scikit-learn, PyTorch) stores its numbers as NumPy arrays underneath
- Arrays are faster than lists because NumPy runs the loop in optimized C code, not in Python

CONCEPT

Creating Arrays

- np.array(list) converts a Python list (or nested list) into a NumPy array
- np.zeros(shape) and np.ones(shape) create arrays pre-filled with 0s or 1s
- np.arange(start, stop, step) works like range(), but returns an array
- np.linspace(start, stop, n) returns n evenly spaced values between start and stop

Array Creation Syntax

SYNTAX

```
import numpy as np

arr = np.array([1, 2, 3])
```

PARAMETERS

np the conventional short name for the numpy module after import

[1, 2, 3] any Python list (or nested list, for 2D and higher) to convert

YOU CAN

✓ Convert a list, or a list of lists, into an array in one line

✓ Build ready-made arrays with np.zeros(), np.ones(), np.arange(), np.linspace()

✓ Mix these with variables, for example np.zeros(len(my_list))

YOU CANNOT

✗ Mix incompatible shapes inside one np.array() call without an error

✗ Change an array's total number of elements just by reassigning .shape

Returns: a new ndarray object (NumPy's array type)

EXAMPLE

Building Arrays a Few Ways

Each function builds a different starting array from scratch.

```
import numpy as np

a = np.array([10, 20, 30])
print(a)              # [10 20 30]

z = np.zeros(4)
print(z)              # [0. 0. 0. 0.]

r = np.arange(0, 10, 2)
print(r)              # [0 2 4 6 8]

lin = np.linspace(0, 1, 5)
print(lin)            # [0.   0.25 0.5  0.75 1.  ]
```

THINK & PRACTICE

Quick Check

Create an array named prices from the list [250, 480, 120, 990] using np.array(). Then create a second array named zeros_5 of five zeros using np.zeros(). Print both.

ANSWER

Quick Check: Answer

```
import numpy as np

prices = np.array([250, 480, 120, 990])
zeros_5 = np.zeros(5)

print(prices)    # [250 480 120 990]
print(zeros_5)   # [0. 0. 0. 0. 0.]
```

CONCEPT

Array Dimensions & Shape

- A 1D array is a flat list of numbers: np.array([1, 2, 3])
- A 2D array is a grid, a list of lists: np.array([[1, 2], [3, 4]])
- .ndim gives the number of dimensions; .shape gives the size along each dimension
- .size gives the total element count; .dtype gives the data type stored (int64, float64, and so on)

EXAMPLE

Inspecting an Array

The same four attributes work on any array, regardless of its shape.

```
import numpy as np

grid = np.array([[1, 2, 3], [4, 5, 6]])

print(grid.ndim)   # 2
print(grid.shape)  # (2, 3) - 2 rows, 3 columns
print(grid.size)   # 6
print(grid.dtype)  # int64
```

THINK & PRACTICE

Quick Check

Create a 2D array called board from [[1, 0, 1], [0, 1, 0]]. Print its .ndim, .shape, and .size.

ANSWER

Quick Check: Answer

```
import numpy as np

board = np.array([[1, 0, 1], [0, 1, 0]])

print(board.ndim)   # 2
print(board.shape)  # (2, 3)
print(board.size)   # 6
```

CONCEPT

Indexing Arrays

- A 1D array indexes just like a list: arr[0], arr[-1]
- A 2D array uses one pair of brackets with a comma: grid[row, col]
- grid[0, 1] means row 0, column 1, not row 0 of column 1
- Negative indices still work in each position: grid[-1, -1] is the bottom-right element

EXAMPLE

Indexing a 2D Array

grid[row, col] reaches one element directly, without a nested loop.

```
import numpy as np

grid = np.array([[10, 20, 30], [40, 50, 60]])

print(grid[0, 0])    # 10 - first row, first column
print(grid[0, 2])    # 30 - first row, last column
print(grid[1, 1])    # 50 - second row, second column
print(grid[-1, -1])  # 60 - last row, last column
```

THINK & PRACTICE

Think & Practice

Create a 2D array temps from [[30, 32, 31], [28, 29, 27]] (row 0 is Karachi, row 1 is Lahore). Print Karachi's middle-day temperature and Lahore's last-day temperature using indexing.

ANSWER

Think & Practice: Answer

```
import numpy as np

temps = np.array([[30, 32, 31], [28, 29, 27]])

print(temps[0, 1])   # 32 - Karachi, middle day
print(temps[1, -1])  # 27 - Lahore, last day
```

THINK & PRACTICE

Think & Practice (Harder)

Create a 3x3 array seats from [[1,1,0],[0,1,1],[1,0,0]] where 1 means booked and 0 means free. Write a function count_booked(seats) that loops over every row and column with indexing (not a NumPy shortcut) and counts how many seats equal 1. Test it on seats.

ANSWER

Think & Practice (Harder): Answer

```
import numpy as np

seats = np.array([[1, 1, 0], [0, 1, 1], [1, 0, 0]])

def count_booked(seats):
    count = 0
    for r in range(seats.shape[0]):
        for c in range(seats.shape[1]):
            if seats[r, c] == 1:
                count += 1
    return count

print(count_booked(seats))  # 4
```

Hour 2

Slicing, Vectorized Operations & Broadcasting

CONCEPT

Slicing Arrays

- 1D slicing works exactly like list slicing: arr[1:4], arr[:3], arr[::2]
- 2D slicing takes a slice for each dimension: grid[row_slice, col_slice]
- grid[:, 1] means every row, column 1 only, a full column
- grid[0, :] means row 0, every column, a full row

EXAMPLE

Slicing Rows and Columns

A colon alone means take everything along that dimension.

```
import numpy as np

grid = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print(grid[0, :])    # [1 2 3] - first row
print(grid[:, 0])    # [1 4 7] - first column
print(grid[0:2, 1:])  # [[2 3] [5 6]] - a sub-grid
```

THINK & PRACTICE

Quick Check

Using scores = np.array([[80, 90], [70, 85], [60, 95]]) (each row is one student's two test scores), print the entire second column (all students' second test score) using slicing.

ANSWER

Quick Check: Answer

```
import numpy as np

scores = np.array([[80, 90], [70, 85], [60, 95]])

print(scores[:, 1])  # [90 85 95]
```

CONCEPT

Vectorized Operations

- A vectorized operation applies to every element at once, with no explicit loop in your code
- arr + 5, arr * 2, arr ** 2 all work directly on the whole array
- Two arrays of the same shape can be added, subtracted, multiplied, or divided element by element
- This is both shorter to write and much faster than looping in plain Python

The Loop Way

```
doubled = []
for n in nums:

doubled.append(n *
2)
```



The Vectorized Way

```
doubled = nums * 2
```

EXAMPLE

Elementwise Math on Arrays

Every operator below applies to all elements in a single step.

```
import numpy as np

nums = np.array([1, 2, 3, 4])

print(nums + 10)   # [11 12 13 14]
print(nums * 2)    # [2 4 6 8]
print(nums ** 2)   # [1 4 9 16]

a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a + b)       # [11 22 33]
```

THINK & PRACTICE

Quick Check

Create bill = np.array([500, 1200, 300]) (three restaurant bills). Compute an array with_tip that adds a 10 percent tip to every bill in one vectorized line, then print it.

ANSWER

Quick Check: Answer

```
import numpy as np

bill = np.array([500, 1200, 300])
with_tip = bill * 1.10

print(with_tip)  # [ 550. 1320.  330.]
```

CONCEPT

Broadcasting

- Broadcasting is how NumPy applies an operation between arrays of different shapes
- A single number (a scalar) broadcasts across an entire array: arr + 5 stretches the 5 to match every element
- A smaller array can broadcast across a larger one if their shapes are compatible, most simply a row matching each row of a 2D array
- If the shapes truly don't fit together, NumPy raises a ValueError instead of guessing

EXAMPLE

Broadcasting a Row Across a Grid

The 1D row array is stretched down to match every row of the 2D grid.

```
import numpy as np

grid = np.array([[1, 2, 3], [4, 5, 6]])
bonus = np.array([100, 200, 300])

print(grid + bonus)
# [[101 202 303]
#  [104 205 306]]
```

THINK & PRACTICE

Think & Practice (Harder)

Create a 2D array marks with 3 students' scores in 2 subjects: [[60, 70], [80, 90], [50, 65]]. Create a 1D array bonus = np.array([5, 10]) (extra marks for subject 1 and subject 2). Use broadcasting to add bonus to every student's row in one line, and print the result.

ANSWER

Think & Practice (Harder): Answer

```
import numpy as np

marks = np.array([[60, 70], [80, 90], [50, 65]])
bonus = np.array([5, 10])

print(marks + bonus)
# [[65 80]
#  [85 100]
#  [55 75]]
```

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

Hour 3

Arrays as the Language of AI Data

CONCEPT

Why AI Data Is Represented as Matrices

- A table of data (rows of examples, columns of features) is naturally a 2D array, a matrix
- A grayscale image is a 2D array of pixel brightness values; a color image adds a third dimension for red, green, blue
- Every machine learning model, from linear regression to a neural network, takes a matrix of numbers in and produces a matrix of numbers out
- This is why NumPy arrays sit underneath Pandas, scikit-learn, and PyTorch: everything eventually becomes numbers in a matrix

CONCEPT

A Quick Note: Views vs. Copies

- Slicing an array (grid[:, 0]) usually returns a view, a window onto the same data, not a separate copy
- Changing a value in a view also changes the original array, which can surprise you
- Use .copy() when you need an independent array: safe_copy = grid.copy()
- This matters most when you pass array slices into functions that modify their input

WORKED EXAMPLE

Cleaning Up a Sensor Reading Grid

Replace every negative reading (a sensor error code) with 0, using vectorized comparison.

```
import numpy as np

readings = np.array([[22, -1, 24], [23, 25, -1]])

cleaned = readings.copy()
cleaned[cleaned < 0] = 0

print(cleaned)
# [[22  0 24]
#  [23 25  0]]
print(readings)   # unchanged, because we used .copy()
```

THINK & PRACTICE

Practice

Create attendance = np.array([[1, 1, 0], [1, 0, 0], [1, 1, 1]]) (rows are students, columns are days, 1 means present). Using a vectorized operation (no loop), compute total_days_present, an array with each student's total present count by summing across each row (hint: use .sum(axis=1)).

ANSWER

Practice: Answer

```
import numpy as np

attendance = np.array([[1, 1, 0], [1, 0, 0], [1, 1, 1]])

total_days_present = attendance.sum(axis=1)
print(total_days_present)  # [2 1 3]
```

THINK & PRACTICE

Practice (Harder)

Create sales = np.array([[100, 200, 150], [300, 100, 400]]) (2 stores, 3 months). Compute: (1) each store's total sales across all months, (2) each month's total sales across both stores, (3) the single highest sale in the whole array. Use .sum(axis=...) and .max().

ANSWER

Practice (Harder): Answer

```
import numpy as np

sales = np.array([[100, 200, 150], [300, 100, 400]])

per_store = sales.sum(axis=1)
print(per_store)     # [450 800]

per_month = sales.sum(axis=0)
print(per_month)     # [400 300 550]

print(sales.max())   # 400
```

Home Practice: Interview-Style Questions

- What is the difference between a Python list and a NumPy array, and why does it matter for speed?
- Explain broadcasting in your own words, using a scalar-plus-array example
- What is the difference between axis=0 and axis=1 in an operation like .sum(axis=...)?
- Why does modifying a slice of an array sometimes change the original array?
- Given a 2D array of exam scores, write one line that returns the average score per student
- Why are NumPy arrays the standard way to represent data across the Python AI ecosystem?

Key Takeaways & Coming Up

- NumPy arrays are fixed-type grids of numbers built for fast, vectorized math
- .shape, .ndim, .size, and .dtype tell you everything about an array's structure
- Indexing and slicing use grid[row, col] with a comma; a colon alone means 'everything' along that axis
- Vectorized operations and broadcasting replace explicit loops, and are the reason NumPy is fast
- Coming up next: Pandas, which builds labeled, spreadsheet-like DataFrames on top of NumPy arrays